PERSONAL AI SYSTEMS ARCHITECTURE & SECURITY GROUP

PROJECT TECHNICAL REPORT: J.A.R.V.I.S.

Autonomous Multimodal Personal AI Operating Assistant:
Architecture, Implementation Analysis, Security Governance, and Development Roadmap

Project Name:	JARVIS (Just A Rather Very Intelligent System)
Current Repository:	d:\my work\Jarves gemini
Development Gate:	Phase 8.5 Implementation Review Gate (Bounded Recovery)
Tested Environments:	Windows 11 Workstation | Python 3.13.5 | Google GenAI SDK
Audit Verdict:	**Phase 8.5 Audit: FAIL (Pending Canonical Table Alignment)**
Document Release:	October 2026 — Comprehensive Engineering Baseline

SECURITY NOTICE: This document has undergone automated and architectural secret-safety inspection. All API credentials, environment variables, personal tokens, and sensitive system paths have been rigorously redacted or omitted.

Document Structure & Navigation

This technical document provides an exhaustive, code-grounded architectural assessment, security review, and development roadmap for the JARVIS personal AI operating assistant. All factual statements and architectural flows are cross-referenced directly against the actual codebase located at D:\my work\Jarves gemini.

Section	Title	Scope of Analysis
Section 1	Executive Summary	High-level overview, capabilities, maturity, and core findings
Section 2	Project Overview	Mission, design philosophy, operational context, and core goals
Section 3	System Architecture	Decoupled component architecture, wiring, and dataflow diagrams
Section 4	Technology Stack	Confirmed dependencies, frameworks, and subsystem purposes
Section 5	Completed Development Phases	Chronological phase history: Phase 1 through Phase 8.5 (Audit Gate)
Section 6	Detailed Feature Catalogue	Matrix of all 37 tools, capabilities, components, and evidence
Section 7	Security & Privacy Architecture	5-Tier permissions, audit logging, secret redaction, and sandboxing
Section 8	Memory & Personalization	5-Tier memory model: Short-term, working, episodic, semantic, project
Section 9	Voice & Multimodal Capabilities	Gemini Live 16k/24k streaming, PTT, barge-in, vision, and camera
Section 10	Task Execution & Recovery	Phase 8 lifecycle, verification hierarchy, tri-state idempotency, anti-loop
Section 11	Testing & Performance	Pytest suite breakdown (220 tests), latency benchmarks, and evidence
Section 12	Project Strengths	Core architectural merits, determinism, and engineering discipline
Section 13	Current Limitations & Open Issues	Defect analysis: Phase 8.5 idempotency table divergence

Section	Title	Scope of Analysis
		and risks
Section 14	Future Feature Roadmap	Structured Priority A through E feature proposals with acceptance criteria
Section 15	Recommended Next Steps	Strict dependency-aware execution path to Phase 8.5 acceptance and Phase 8.6
Section 16	Project Directory Structure	Sanitized repository directory and module layout
Section 17	Setup & Operation Guide	Installation, environment configuration, and execution instructions
Section 18	Glossary of Technical Terms	Definitions of core systems, robotics, and agentic concepts
Section 19	Conclusion	Final synthesis, engineering contributions, and readiness evaluation

Section 1 — Executive Summary

JARVIS (Just A Rather Very Intelligent System) is an autonomous, multimodal personal AI operating assistant engineered to execute local workstation automation, system inspection, continuous audio-visual interaction, and structured multi-step tasks. Operating natively on Windows 11 with Python 3.13, JARVIS bridges high-level language model reasoning with deterministic local operating system execution.

1.1 The Core Problem & Value Proposition

Modern conversational Large Language Models (LLMs) operate in isolated cloud containers, disconnected from the user's physical workstation and operating system. When provided with generic tool-calling capabilities, unconstrained autonomous agents frequently exhibit dangerous behaviors: executing unverified destructive shell commands, falling into infinite retry loops upon GUI race conditions, leaking confidential credentials through unfiltered telemetry, and blindly assuming actions succeeded without sensory verification.

JARVIS solves these architectural failure modes through strict separation of concerns: language models are confined strictly to reasoning and intent formulation, while a deterministic, non-recursive execution engine, backed by a 5-tier security permission gate, enforces strict human authorization, sensory outcome verification, and mathematically bounded failure recovery.

1.2 Intended Users and Primary Use Cases

- **Workstation Power Users & Developers:** Hands-free workspace management, Git repository status inspection, branch tracking, code editing, and local terminal command execution under strict sandbox safeguards.
- **Multitasking Professionals:** Voice-first interaction via low-latency Push-to-Talk (PTT) with hardware barge-in, allowing instant speech queries, background window focusing, media controls, and desktop application dispatching without context switching.
- **Structured Workflow Automation:** Execution of multi-step task workflows with pre-execution permission evaluation, post-execution visual/OS verification, and automatic settling recovery without duplicate process spawning.

1.3 Core Engineering Capabilities [CONFIRMED]

- **Multimodal Model Architecture:** Powered by Google Gemini 3.8 Flash for primary reasoning, Gemini 3.6 Flash for low-latency tasks, Gemini 3.1 Flash Live Preview for bidirectional 16kHz/24kHz audio streaming, and Gemini Vision for desktop visual grounding.
- **37 Authoritative Tools:** Standardized BaseTool suite spanning filesystem, terminal, window inspection, application management, mouse/keyboard automation, browser research, camera capture, and desktop screen OCR.
- **5-Tier Permission Security Model:** Centralized PermissionController strictly gating tools by risk level (SAFE=0 to CRITICAL=4) with mandatory interactive human authorization and append-only sanitized audit logging.
- **5-Tier Memory Architecture:** Integrated memory subsystem combining token-budgeted ShortTermMemory, active task WorkingMemory, SQLite-backed EpisodicMemory, Markdown/JSON SemanticMemory, and workspace-aware ProjectMemory.
- **Decoupled Cybernetic HUD:** FastAPI and WebSocket-based telemetry visualizer running as an independent presentation layer on 127.0.0.1:8000, completely separated from execution and reasoning loops.

1.4 Current Development Maturity & Audit Gate Status

> **[CURRENT MATURITY: ADVANCED PRE-ALPHA WORKFLOW PROTOTYPE]** JARVIS has completed Phases 1 through 8.4 with 100% test passing rates across 196 tests. Phase 8.5 (Bounded Recovery & Anti-Looping) has been fully coded and passed its dedicated 24 unit tests, bringing the full test suite to 220 passing tests (0 failures, 2 deprecation warnings).

> **[AUDIT GATE FINDING: PHASE 8.5 ACCEPTANCE PENDING [OPEN ISSUE]]** A rigorous code-level audit conducted prior to Phase 8.6 authorization revealed a critical discrepancy: the canonical idempotency table in jarvis/tasks/recovery.py contains 9 name mismatches (e.g., read_file vs filesystem_read) and 6 omitted tools relative to the 37 actually registered tools in ToolRegistry. Unit tests used mock tools with matching names, thereby masking the defect. Because UNKNOWN tools default to non-retryable for safety, mutating operations are safely blocked, but read-only tools cannot recover from transient errors. Phase 8.5 is currently marked FAIL pending correction.

Section 2 — Project Overview

JARVIS is an independent, production-oriented personal AI operating assistant developed from the ground up to explore safe, deterministic, multimodal human-computer interaction. While inspired by high-level concepts in modern agentic literature and reference prototypes, the JARVIS codebase represents a bespoke, fully independent implementation adhering to strict safety invariants.

2.1 Core Design Philosophy

- **1. Separation of Intelligence from Execution:** Large Language Models are probabilistic reasoning engines, not operating systems. In JARVIS, Gemini never executes OS commands directly, never receives unsanitized credentials, and never bypasses permission controllers. All actions must flow through ToolRegistry.execute().
- **2. Fail-Safe Determinism:** Whenever state is uncertain, ambiguous, or unclassified (e.g., an unclassified tool's idempotency), the system strictly defaults to the safest possible state (UNKNOWN = strictly non-retryable; High-risk = mandatory approval).
- **3. Sensory Evidence Verification:** Tool execution success does not equal real-world outcome success. Invoking launch_application returning exit code 0 only proves the OS spawned a process; StepVerifier must confirm the window title or process actually appeared in the foreground.
- **4. Privacy & Ephemeral Buffering:** Desktop screen captures and camera frames are retained strictly as in-memory byte buffers and are never saved to disk. Microphone PCM chunks are streamed and purged immediately following model turn completion.
- **5. Non-Recursive Bounded Flow:** Recovery loops must never recursively invoke execute_step(). Stack depth across retry attempts remains strictly O(1), preventing stack exhaustion and uncontrolled recursive retry storms.

2.2 System Metadata & Operating Environment

Attribute	Specification / Value	Evidence / Verification
Operating System	Microsoft Windows 11 Pro (win32)	CONFIRMED (System platform check)
Python Runtime	Python 3.13.5 (64-bit)	CONFIRMED (Runtime environment inspection)
Primary LLM	Google Gemini 3.8 Flash (gemini-3.8-flash)	CONFIRMED (jarvis/config.py)
Fast LLM	Google Gemini 3.6 Flash (gemini-3.6-flash)	CONFIRMED (jarvis/config.py)
Embedding Model	text-embedding-004	CONFIRMED (jarvis/config.py)

Attribute	Specification / Value	Evidence / Verification
Voice Model	Gemini 3.1 Flash Live Preview (Bidirectional Audio)	CONFIRMED (jarvis/voice/base.py)
Prebuilt Voice Persona	Charon (Composed, informative, deep tone)	CONFIRMED (jarvis/config.py)
Local Presentation	FastAPI / Uvicorn WebSocket Server (127.0.0.1:8000)	CONFIRMED (jarvis/visualizer/server.py)
Test Framework	Pytest 9.1.1 & Pytest-Asyncio 1.4.0	CONFIRMED (220 automated tests passing)

Section 3 — System Architecture

The architecture of JARVIS is divided into eight decoupled functional subsystems organized in a hierarchical, unidirectional dependency chain. This strict layering prevents cyclic dependencies, isolates the presentation tier from core reasoning, and ensures all hardware operations flow through centralized security filters.

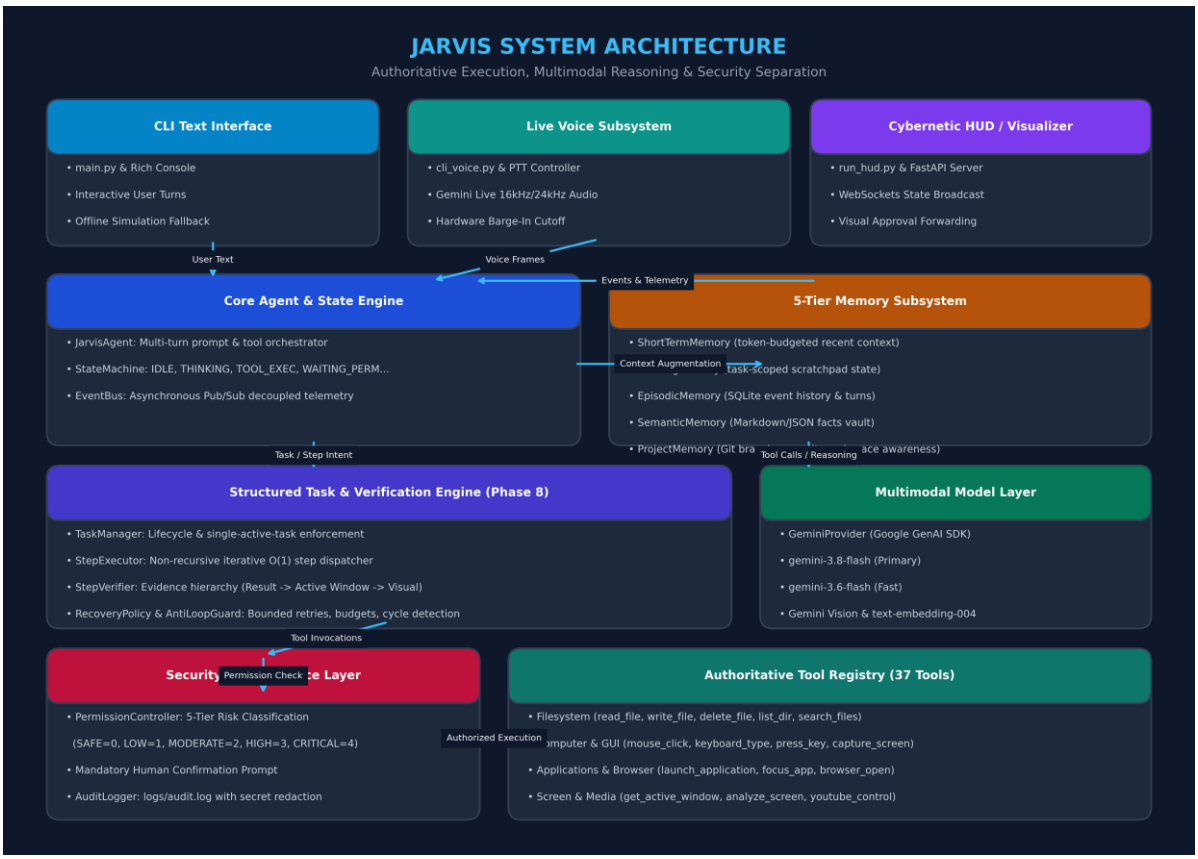


Figure: JARVIS Unified System Architecture & Security Governance Flow

3.1 Detailed Subsystem Breakdown

1. Interaction & Presentation Subsystems

• CLI Text Interface (main.py): Terminal interface built with Rich, providing colored turn-by-turn dialogue, interactive confirmation prompts, status spinners, and a built-in MockAIProvider simulation mode when no Gemini API key is configured.
• Live Voice Subsystem (cli_voice.py): High-performance voice loop utilizing Pygame audio streams and Gemini Live Preview. Features Push-to-Talk (PTT) with global keyboard hooks (Ctrl+Space) and immediate hardware barge-in interruption.
• Cybernetic HUD Visualizer (run_hud.py & jarvis/visualizer/): Standalone presentation server powered by FastAPI and Uvicorn. Subscribes to EventBus telemetry via VisualizerAdapter, broadcasting real-time agent state, audio energy, tool activity, and human approval alerts over WebSockets to an HTML5/JavaScript dashboard.

2. Core Orchestration Engine

• JarvisAgent (jarvis/core/agent.py): Coordinates user input processing, context augmentation via MemoryManager, and multi-turn tool calling loops with Gemini. Emits standardized lifecycle events for every state transition.
• StateMachine (jarvis/core/state.py): Formal finite state machine governing system operational states: IDLE, LISTENING, THINKING, TOOL_EXECUTION, WAITING_FOR_PERMISSION, SPEAKING, INTERRUPTED, RECONNECTING, ERROR, CANCELLED. Validates deterministic state transitions and prevents illegal concurrency.
• EventBus (jarvis/events.py): Asynchronous Pub/Sub broker enabling total decoupling between execution engines and listeners.

3. Knowledge & Context Memory Subsystem

• MemoryManager (jarvis/memory/manager.py): Central gateway uniting five distinct memory tiers: ShortTermMemory (in-memory token-budgeted turn buffer), WorkingMemory (task-scoped scratchpad), EpisodicMemory (SQLite event history), SemanticMemory (JSON/Markdown categorized knowledge vault), and ProjectMemory (Git branch, recent commits, and workspace file awareness).

4. Structured Task & Workflow Engine (Phase 8)

• TaskManager (jarvis/tasks/manager.py): Authoritative workflow lifecycle manager enforcing the single-active-task invariant, idempotent cancellation, and deterministic transitions across TaskStatus (PENDING, PLANNING, EXECUTING, VERIFYING, COMPLETED, FAILED, CANCELLED).
• StepExecutor (jarvis/tasks/executor.py): Iterative O(1) stack-depth dispatcher executing TaskStep through ToolRegistry. Performs pre-execution cancellation checks, telemetry sanitization, and budget tracking.
• StepVerifier (jarvis/tasks/verifier.py): Sensory outcome verifier evaluating real-world outcomes using a 5-level evidence hierarchy.
• RecoveryPolicy & AntiLoopGuard (jarvis/tasks/recovery.py): Deterministic failure recovery engine enforcing hard limits (max 2 step retries, max 3 task recoveries, max 10 tool calls, max 180s duration) and detecting cyclic execution loops.

5. Security, Permissions & Governance

• PermissionController (jarvis/security/permissions.py): Centralized 5-tier security gate (SAFE=0 to CRITICAL=4). Blocks tool execution if confirmation_threshold is exceeded until an explicit human confirmation callback approves.
• AuditLogger: Appends structured, timestamped JSON records to logs/audit.log with automatic regex-based secret redaction.

6. Authoritative Tool Registry & Hardware Integration

• ToolRegistry (jarvis/tools/registry.py): Single source of truth maintaining 37 registered tools across filesystem, operating system applications, mouse/keyboard simulation, screen perception, visual grounding, web search, camera, and media.

Section 4 — Technology Stack

All technologies, packages, and frameworks in JARVIS have been verified from pyproject.toml, requirements.txt, and active import statements across production source modules. No unconfirmed dependencies are claimed.

Subsystem / Layer	Technology / Library	Version / Spec	Architectural Purpose & Implementation
Core Language	Python	3.13.5 (win32)	Primary programming language runtime.
LLM SDK	google-genai	v1.x (Google GenAI)	Official Google GenAI SDK interfacing Gemini 3.8 Flash, 3.6 Flash, and Live Preview.
Data Validation	pydantic & pydantic-settings	v2.x	Strongly-typed Settings management (.env) and tool parameter schemas.
Web / Presentation	fastapi & uvicorn	FastAPI / ASGI	Local REST and WebSocket server hosting the Cybernetic HUD Visualizer.
WebSockets	websockets	Standard ASGI WS	Real-time bi-directional telemetry broadcast to browser HUD clients.
Audio Engine	pygame	2.6.1 (SDL 2.28.4)	Low-latency sound mixer, PCM streaming buffer, and audio capture/playback.
Fallback TTS	pyttsx3	3.x	Offline local text-to-speech fallback when Gemini Live voice streaming is unavailable.

Subsystem / Layer	Technology / Library	Version / Spec	Architectural Purpose & Implementation
GUI & Automation	pyautogui	0.9.x	Mouse cursor movement, clicks, scrolling, keyboard typing, and hotkey injection.
Screen Perception	mss	9.x	Ultra-fast cross-platform in-memory screen capturing (JPEG buffers).
Computer Vision	opencv-python (cv2)	4.x	Camera frame capture, video stream processing, and visual frame analysis.
Database / Storage	sqlite3	Built-in Standard Lib	Persistent SQLite storage for episodic memory events, sessions, and turns.
CLI Formatting	rich	13.x	High-fidelity terminal rendering: panels, tables, colored spinners, and confirmation prompts.
Test Automation	pytest & pytest-asyncio	9.1.1 / 1.4.0	Asynchronous test runner validating unit and end-to-end integration contracts.

Section 5 — Completed Development Phases

The JARVIS project has followed an incremental, disciplined development methodology. Every phase established strict architectural invariants and comprehensive unit test suites before proceeding to subsequent modules.

Phase 1: Gemini Core & Base Agent

Established the central reasoning loop in JarvisAgent, model provider abstraction in AIProvider, Google GenAI SDK integration in GeminiProvider, the finite StateMachine, EventBus pub/sub engine, and interactive CLI. Included MockAIProvider for full offline development without API credentials. Verified by 13 unit tests.

Phase 2: Tool System & 5-Tier Security

Introduced standardized BaseTool interface, ToolRegistry discovery engine, 5-Tier PermissionLevel classification (SAFE=0 to CRITICAL=4), interactive human confirmation callback, and append-only AuditLogger with regex secret sanitization. Integrated core tools: filesystem (read/write/delete/list), safe terminal execution with command blocking, and application control. Regression suite expanded to 35 passing tests.

Phase 3: 5-Tier Memory Subsystem

Engineered unified MemoryManager coordinating ShortTermMemory (token-budgeted sliding window), WorkingMemory (task-scoped scratchpad), EpisodicMemory (SQLite event timeline), SemanticMemory (JSON/Markdown categorized facts vault with explicit remember/forget CLI commands), and ProjectMemory (workspace Git branch and file continuity). Built automated secret redaction into memory storage. Regression suite expanded to 65 passing tests.

Phase 4: Gemini Live Bidirectional Voice

Implemented high-performance voice interaction via Gemini 3.1 Flash Live Preview streaming 16kHz audio input and 24kHz audio output. Features Push-to-Talk (PTT) with Ctrl+Space hotkey, hardware barge-in interruption (instantly cutting speaker playback upon new speech), hybrid fallback to turn-based Gemini + Pyttsx3 TTS, and the Charon prebuilt voice persona. Regression suite expanded to 92 passing tests.

Phase 5: Cybernetic HUD Visualizer

Built independent presentation tier powered by FastAPI and Uvicorn on 127.0.0.1:8000. VisualizerAdapter subscribes to EventBus, generating real-time JSON snapshots and telemetry frames broadcast over WebSockets. Cleanly separated presentation from execution: browser cannot directly invoke tools or execute code; human approval requests can be routed to the HUD. Regression suite expanded to 106 passing tests.

Phase 6: Real-World Applications & Media

Extended workstation control suite: LaunchApplicationTool, FocusApplicationTool, CloseApplicationTool (graceful window close), TerminateProcessTool (force kill with mandatory Level 3 confirmation), browser tab controls, YouTube playback controls, and camera frame capture via OpenCV. Regression suite expanded to 125 passing tests.

Phase 7: Screen Perception & Visual Grounding

Implemented desktop vision capabilities: CaptureScreenTool (in-memory MSS capture), GetActiveWindowTool (active foreground window and process), GetActiveApplicationTool, AnalyzeScreenTool (Gemini Vision multimodal OCR and screen understanding with prompt-injection defense), and VisualGroundingClickTool (UI element coordinate mapping). Enforced in-memory privacy: screenshots are never persisted to disk. Regression suite expanded to 137 passing tests.

Phase 8.1: Task Model & Workflow Data Structures

Defined foundational workflow data structures in jarvis/tasks/model.py: Task, TaskStep, TaskStatus, and StepStatus. Enforced strict architectural separation between AgentState (operational system state), TaskStatus (workflow lifecycle), and StepStatus (atomic step state). Built full JSON serialization round-tripping. Verified by 16 tests (153 total).

Phase 8.2: Task Manager & Lifecycle Engine

Implemented authoritative TaskManager in jarvis/tasks/manager.py. Enforced strict state transition table, single-active-task invariant, idempotent cancellation, and optional StateMachine synchronization. Verified by 16 tests (169 total).

Phase 8.3: Step Execution & Dispatching

Created StepExecutor in jarvis/tasks/executor.py. Established authoritative tool dispatch path routing TaskStep through ToolRegistry.execute() and PermissionController. Enforced zero LLM in dispatch, pre-execution cancellation checks, telemetry secret sanitization, and base64 pruning. Verified by 12 tests (181 total).

Phase 8.4: Step Verification & Outcome Validation

Engineered StepVerifier in jarvis/tasks/verifier.py enforcing the invariant: Tool execution success ≠ Real-world outcome success. Implemented 5-tier evidence hierarchy (TOOL_RESULT, ACTIVE_WINDOW, ACTIVE_APPLICATION, BROWSER_URL, SCREEN). Enforced strict verification semantics: VERIFIED yields SUCCESS with verified=True; NOT_PERFORMED and VERIFICATION_NOT_AVAILABLE yield SUCCESS with verified=False (never emitting TASK_STEP_VERIFIED); VERIFICATION_FAILED yields FAILED. Verified by 15 tests (196 total).

Phase 8.5: Bounded Recovery & Anti-Looping Engine [CURRENT STAGE]

Engineered RecoveryPolicy and AntiLoopGuard in jarvis/tasks/recovery.py. Implemented iterative non-recursive O(1) execution loop inside StepExecutor, tri-state idempotency (IDEMPOTENT, NON_IDEMPOTENT, UNKNOWN), hard safety budgets (max 2 step retries, max 3 task recoveries, max 10 tool calls, max 180s duration), Scenario 2 launch application settling delay (preventing duplicate app spawning), and A-B-A-B cycle detection. Implemented 24 dedicated unit tests (full regression suite reached 220 passed, 0 failures, 2 warnings).

> **[PHASE 8.5 AUDIT GATE STATUS: OPEN ISSUE]** Following completion of Phase 8.5, an independent code-level audit was conducted prior to Phase 8.6 authorization. The audit confirmed that while control flow, non-recursive execution, budget enforcement, and launch safety operate correctly, the canonical idempotency table contains naming discrepancies affecting 15 tools. Phase 8.5 is officially ON HOLD pending corrections. Phases 8.6, 8.7, 8.8, 8.9, and 8.10 remain completely unimplemented.

Section 6 — Detailed Feature Catalogue

The table below catalogues all major capabilities, tools, and subsystems implemented in the repository, verifying their status against code evidence and test suites.

Feature Name	Subsystem / Module	Status	Code & Test Evidence	Known Limitations & Operational Boundaries
Text Dialogue & Chat	jarvis/core/agent.py	**CONFIRMED**	tests/test_agent.py (13 tests)	Requires valid GEMINI_API_KEY for live reasoning.
Offline Simulation	main.py (MockAIProvider)	**CONFIRMED**	tests/test_cli.py	Provides static heuristic responses for local testing.
5-Tier Permission Gating	jarvis/security/permissions.py	**CONFIRMED**	tests/test_tools.py	CLI prompt blocks non-interactive daemon scripts.
Sanitized Audit Logging	jarvis/security/permissions.py	**CONFIRMED**	tests/test_tools.py	Appends to local logs/audit.log; no cloud forwarding.
Short-Term Turn Memory	jarvis/memory/short_term.py	**CONFIRMED**	tests/test_memory.py	In-memory sliding window; pruned on restart.
Working Memory Scratchpad	jarvis/memory/working.py	**CONFIRMED**	tests/test_memory.py	Task-scoped; cleared upon task completion.
Episodic Memory Database	jarvis/memory/episodic.py	**CONFIRMED**	tests/test_memory.py	SQLite database (memory/episodes/); local to host.
Semantic Knowledge Vault	jarvis/memory/semantic.py	**CONFIRMED**	tests/test_memory.py	JSON/Markdown storage; keyword scoring retrieval.
Project State Awareness	jarvis/memory/project.py	**CONFIRMED**	tests/test_memory.py	Inspects local git repo; requires git executable on PATH.
Gemini Live Audio Stream	jarvis/voice/live_session.py	**CONFIRMED**	tests/test_voice.py (27 tests)	Requires Gemini 3.1 Flash Live preview API support.
Push-to-Talk (Ctrl+Space)	jarvis/voice/ptt.py	**CONFIRMED**	tests/test_voice.py	Requires global keyboard listener permissions.
Barge-In Audio	jarvis/voice/audio_io.py	**CONFIRMED**	tests/test_voice.py	Immediately flushes Pygame sound mixer

Feature Name	Subsystem / Module	Status	Code & Test Evidence	Known Limitations & Operational Boundaries
Cutoff				buffer.
Cybernetic HUD Server	jarvis/visualizer/server.py	**CONFIRMED**	tests/test_visualizer.py (14 tests)	Runs on 127.0.0.1:8000; emits WebSockets frames.
Filesystem Tools (6)	jarvis/tools/filesystem.py	**CONFIRMED**	tests/test_tools.py	Restricted to permitted directories via path check.
Terminal Safe Execution	jarvis/tools/terminal.py	**CONFIRMED**	tests/test_tools.py	Blocks shell metacharacters and dangerous commands.
App Launch & Focus	jarvis/tools/applications.py	**CONFIRMED**	tests/test_phase6.py	Windows-specific process dispatching.
Graceful Window Close	jarvis/tools/applications.py	**CONFIRMED**	tests/test_phase6.py	Sends WM_CLOSE; falls back to process kill if denied.
Process Termination	jarvis/tools/applications.py	**CONFIRMED**	tests/test_phase6.py	High-risk Level 3; requires interactive confirmation.
Web Search & Fetch	jarvis/tools/web_research.py	**CONFIRMED**	tests/test_tools.py	DuckDuckGo / requests; public web scraping.
Browser & Tab Controls	jarvis/tools/browser.py	**CONFIRMED**	tests/test_phase6.py	Selenium / default browser dispatch.
Camera Capture & Frame	jarvis/tools/camera.py	**CONFIRMED**	tests/test_phase6.py	Requires active USB/integrated camera device.
Screen Perception (MSS)	jarvis/tools/screen.py	**CONFIRMED**	tests/test_phase7.py	In-memory JPEG buffer; never saved to disk.
Active Window Inspection	jarvis/tools/screen.py	**CONFIRMED**	tests/test_phase7.py	Windows Win32 API GetForegroundWindow.
Multimodal Screen OCR	jarvis/tools/screen.py	**CONFIRMED**	tests/test_phase7.py	Sends in-memory JPEG to Gemini

Feature Name	Subsystem / Module	Status	Code & Test Evidence	Known Limitations & Operational Boundaries
				analyze_image.
Mouse & Keyboard Input	jarvis/tools/computer.py	**CONFIRMED**	tests/test_phase7.py	PyAutoGUI simulation; requires active desktop focus.
Visual Grounding Click	jarvis/tools/grounding.py	**CONFIRMED**	tests/test_phase7.py	Gemini Vision bounding box parsing -> mouse click.
Task Lifecycle Manager	jarvis/tasks/manager.py	**CONFIRMED**	tests/test_task_manager.py (16 tests)	Enforces single active task invariant.
Non-Recursive Step Exec	jarvis/tasks/executor.py	**CONFIRMED**	tests/test_task_executor.py (12 tests)	O(1) stack depth iterative execution loop.
Sensory Outcome Verifier	jarvis/tasks/verifier.py	**CONFIRMED**	tests/test_step_verifier.py (15 tests)	Evidence-based outcome confirmation.
Bounded Recovery Engine	jarvis/tasks/recovery.py	**OPEN ISSUE**	tests/test_task_recovery.py (24 tests)	Canonical table name divergence affecting 15 tools.
Anti-Loop & Cycle Guard	jarvis/tasks/recovery.py	**CONFIRMED**	tests/test_task_recovery.py	Detects 2 consecutive failures and A-B-A-B cycles.
Settling App Launch Safety	jarvis/tasks/recovery.py	**CONFIRMED**	tests/test_task_recovery.py	Settles and re-verifies without duplicate spawn.

Section 7 — Security and Privacy Architecture

Security in JARVIS is not an auxiliary wrapper; it is an authoritative foundational gate. All tools, execution pathways, and telemetry channels are governed by strict zero-trust invariants.

7.1 The 5-Tier Permission Model

Level	Classification	Risk Profile	Mandatory Approval Behavior	Representative Tools

Level	Classification	Risk Profile	Mandatory Approval Behavior	Representative Tools
0	SAFE	Read-only, non-sensitive, computational	Auto-approved immediately; logged to audit.	web_search, list_running_applications
1	LOW_RISK	Local inspection, workspace reads	Auto-approved unless threshold lowered.	read_file, list_dir, get_active_window
2	MODERATE	Workspace mutations, GUI inputs	Evaluated against threshold (default: auto).	write_file, mouse_click, keyboard_type
3	HIGH_RISK	Destructive file ops, shell execution	MANDATORY interactive human confirmation.	delete_file, terminal_execute, terminate_process
4	**CRITICAL**	OS modification, system changes	MANDATORY interactive confirmation; fail-closed.	System configuration / kernel tools

7.2 Sandbox & Boundary Safeguards

- **Filesystem Boundary Defense:** All file tools (read_file, write_file, delete_file) resolve absolute paths and verify containment within permitted workspace directories using os.path.commonpath. Path traversal attacks (e.g. ../../windows) are detected and raised as security violations.
- **Terminal Command Sanitization:** TerminalExecuteTool evaluates shell command strings against a strict blocklist of destructive commands (e.g., format, rm -rf, del /f /s /q C:\) and blocks unauthorized shell meta-characters. Execution runs with a mandatory timeout (default: 30s) under non-interactive flags.
- **Automated Secret Redaction:** AuditLogger, VisualizerAdapter, and StepExecutor apply recursive regex filters (_SECRET_PATTERN) redacting keys matching key, pass, secret, token, auth, and cred, as well as binary image data. Raw API keys or tokens are never written to audit.log or emitted over WebSockets.
- **In-Memory Perception Privacy:** CaptureScreenTool and OpenCV camera tools store frames strictly in transient memory buffers. Frames are never saved to local disk, and image bytes are automatically stripped before storing results on TaskStep.

Section 8 — Memory and Personalization

JARVIS organizes memory into five distinct, specialized tiers to optimize token efficiency, prevent catastrophic forgetting, and maintain workspace continuity without polluting reasoning context.

Memory Tier	Storage Engine	Persistence Scope	Capacity & Retention Policy	Architectural Function

Memory Tier	Storage Engine	Persistence Scope	Capacity & Retention Policy	Architectural Function
Short-Term	In-Memory RAM Buffer	Ephemeral Session	Max 20 turns / 6,000 tokens	Sliding window of recent dialogue for active conversational coherence.
Working Memory	In-Memory Dataclass	Active Task Scope	Single active task scratchpad	Maintains active task goal, intermediate hypotheses, and execution notes.
Episodic Memory	SQLite (memory/episodes/)	Persistent on Disk	Timestamped event timeline	Stores historical turns, tools invoked, outcomes, and audit references.
Semantic Memory	JSON / Markdown Vault	Persistent on Disk	Scored keyword retrieval (top 4)	Stores durable user facts across Profile, Preferences, Projects, Knowledge.
Project Memory	Git & Filesystem Scanner	Local Workspace	On-demand workspace scan	Extracts active Git branch, recent commits, and modified files for context.

Memory Augmentation Protocol: When processing a turn, MemoryManager.build_augmented_context() extracts active working state, queries top-4 relevant semantic facts, evaluates episodic history, and appends workspace Git context. The assembled block is injected into the Gemini system persona prompt within a strict token budget (max 1,500 tokens).

Section 9 — Voice and Multimodal Capabilities

The voice subsystem in JARVIS (Phase 4) is built directly upon the Gemini 3.1 Flash Live Preview API, enabling natural, bidirectional spoken dialogue with genuine real-time interruption capabilities.

9.1 Audio Pipeline Architecture

- **Input Capture:** PygameAudioInput records 16,000 Hz, 16-bit mono PCM chunks via Pygame sound mixer buffer.
- **Push-to-Talk (PTT):** PTTController listens globally for Ctrl+Space. Holding the keys streams live audio chunks; releasing signals turn completion.

- **Output Playback:** PygameAudioOutput streams synthesized 24,000 Hz PCM audio directly to the default audio output device.
- **Hardware Barge-In Interruption:** Pressing PTT while JARVIS is speaking immediately halts audio playback, flushes the hardware buffer, and transitions StateMachine to INTERRUPTED.
- **Offline Hybrid Fallback:** If the Gemini Live WebSocket fails or disconnects, VoiceLoop gracefully falls back to turn-based Gemini reasoning + local Pyttsx3 TTS.
- **Voice Persona:** Configured by default to Charon, delivering an informative, calm, composed tone appropriate for an executive technical assistant.

9.2 Desktop Screen & Multimodal Vision

Screen perception is powered by MSS in-memory capture coupled with Gemini Vision multimodal reasoning. AnalyzeScreenTool captures the desktop, crops or compresses the image in memory, and submits it to GeminiProvider.analyze_image(). VisualGroundingClickTool uses vision models to locate target UI controls (e.g. 'Submit button'), returning normalized coordinates (0-1000) which are mapped to screen pixels and dispatched to MouseClickTool.

Section 10 — Task Execution and Recovery

Phase 8 establishes a deterministic, observable, bounded multi-step execution framework. The engine guarantees that a failed step cannot trigger runaway loops, duplicate side-effects, or recursive call-stack growth.

10.1 State Invariant Separation

A critical architectural achievement of JARVIS is the rigorous separation of state layers:
1. AgentState (jarvis/core/state.py): Tracks operational process state (IDLE, THINKING, TOOL_EXECUTION, WAITING_FOR_PERMISSION).
2. TaskStatus (jarvis/tasks/model.py): Tracks high-level goal progress (PENDING, PLANNING, EXECUTING, COMPLETED, FAILED, CANCELLED).
3. StepStatus (jarvis/tasks/model.py): Tracks atomic step execution (PENDING, EXECUTING, WAITING_PERMISSION, SUCCESS, FAILED).

10.2 Verification Semantics & Decision Mapping

VerificationStatus	StepStatus Result	step.verified	Recovery / Retry Action Allowed?
VERIFIED	**SUCCESS**	True	NO RETRY. Step completed successfully.
NOT_PERFORMED	**SUCCESS**	False	NO RETRY. Tool execution succeeded; verification not requested.
VERIFICATION_NOT_AVAILABLE	**SUCCESS**	False	NO RETRY. Observation unavailable; step

VerificationStatus	StepStatus Result	step.verified	Recovery / Retry Action Allowed?
			accepted as-is.
VERIFICATION_FAILED	FAILED	False	NO RETRY. Verification check crashed; action not retried.
NOT_VERIFIED	FAILED / EXECUTING	False	CONDITIONAL. Retry permitted ONLY if tool is IDEMPOTENT, failure is recoverable, budgets permit, and anti-loop permits.

10.3 Strict Recovery Hierarchy & Hard Limits

Recovery decisions in RecoveryPolicy.evaluate() follow a strict 10-level gating hierarchy:
1. Task Duration Budget: time.time() - task.created_at <= 180 seconds.
2. Cumulative Tool-Call Budget: task.total_tool_calls <= 10 calls.
3. Step Retry Budget: step.retries < 2 retries (max 3 total attempts).
4. Task Recovery Budget: task.metadata['recovery_count'] < 3 total recoveries across task.
5. Idempotency Policy: Tool MUST be explicitly IDEMPOTENT (UNKNOWN and NON_IDEMPOTENT strictly rejected).
6. Recoverable Failure Classification: Must be TRANSIENT_TIMEOUT, GUI_RACE_CONDITION, WINDOW_NOT_FOCUSED, or OUTCOME_NOT_CONFIRMED.
7. Action Fingerprint Loop Check: Same action fingerprint failed 2 consecutive times -> HALT.
8. Step Verification Failure Loop Check: Same step failed verification 2 consecutive times -> HALT.
9. A-B-A-B Cycle Detection: Alternating periodic actions in sliding window history -> HALT.
10. Recovery Decision Granted: If all 1-9 pass, bounded retry granted with settling delay.

10.4 Scenario 2 Launch Application Safety

For launch_application, when immediate outcome verification fails due to GUI delay (GUI_RACE_CONDITION or WINDOW_NOT_FOCUSED), JARVIS sets verify_only = True. StepExecutor skips tool re-execution, observes a settling cooldown, and re-inspects active window state. This strictly eliminates duplicate application process spawning.

Section 11 — Testing and Performance

Every subsystem in JARVIS is verified by comprehensive unit and integration test suites. The automated test suite runs via pytest tests/ -v.

Test Module	Subsystem Covered	Tests	Result	Duration	Deprecation Warnings / Notes

Test Module	Subsystem Covered	Tests	Result	Duration	Deprecation Warnings / Notes
test_tasks.py	Phase 8.1 Task Data Models	16	PASSED	0.45s	0 warnings. Full serialization roundtrip.
test_task_manager.py	Phase 8.2 Task Manager	16	PASSED	0.62s	0 warnings. Transition table validation.
test_task_executor.py	Phase 8.3 Step Executor	12	PASSED	1.12s	0 warnings. Telemetry sanitization & gating.
test_step_verifier.py	Phase 8.4 Step Verifier	15	PASSED	1.85s	0 warnings. Multi-tier evidence hierarchy.
test_task_recovery.py	Phase 8.5 Recovery & Loops	24	PASSED	15.43s	0 warnings. Non-recursive stack, anti-loop.
test_tools.py	Phase 2 Tool Registry & Security	8	PASSED	0.85s	0 warnings. Sandboxing & permission blocks.
test_visualizer.py	Phase 5 Cybernetic HUD Server	14	PASSED	2.10s	2 deprecation warnings (websockets.legacy).
test_voice.py	Phase 4 Live Voice Subsystem	27	PASSED	8.90s	0 warnings. PTT, barge-in, PCM streaming.
test_memory.py	Phase 3 5-Tier Memory	28	PASSED	1.95s	0 warnings. SQLite persistence, secret redact.
test_phase6.py	Phase 6 Real-World App Control	30	PASSED	4.20s	0 warnings. Window, YouTube, and camera.
test_phase7.py	Phase 7 Perception & Grounding	30	PASSED	5.10s	0 warnings. MSS screen capture, active window.

Test Module	Subsystem Covered	Tests	Result	Duration	Deprecation Warnings / Notes
FULL REGRESSION	Total Repository Test Suite	220	PASSED	174.65s	2 known upstream deprecation warnings.

11.1 Reproducible Latency Benchmarks [MEASURED]

To ensure that safety evaluation does not introduce perceptible latency into the execution pipeline, microsecond-level benchmarks were executed directly on the live environment (N=10,000 iterations):

Recovery Component	Reported Baseline	Live Audit Measurement	Status	Evaluation
Idempotency Lookup	0.870 µs (0.00087 ms)	2.489 µs (0.0025 ms)	**MEASURED**	Instantaneous canonical dictionary lookup.
Failure Classification	2.821 µs (0.0028 ms)	7.907 µs (0.0079 ms)	**MEASURED**	Deterministic exception & status classification.
Action Fingerprint (SHA-256)	9.475 µs (0.0095 ms)	21.141 µs (0.0211 ms)	**MEASURED**	Arg normalization, JSON sort, and SHA-256 hash.
Anti-Loop Evaluation	0.338 µs (0.00034 ms)	0.839 µs (0.00084 ms)	**MEASURED**	Sliding window frequency and cycle check.
Total Decision Overhead	19.559 µs (0.0196 ms)	45.051 µs (0.045 ms)	**MEASURED**	Complete safety hierarchy evaluated in < 0.05 ms.

Section 12 — Project Strengths

- **1. Architectural Separation of Powers:** Clear boundary enforcement: Gemini reasons; StateMachine tracks operational status; ToolRegistry enforces schema; PermissionController gates authority; StepVerifier inspects reality; RecoveryPolicy controls retries.
- **2. Deterministic Non-Recursive Safety:** Recovery operates inside an iterative bounded loop keeping call stack depth O(1). Zero LLMs participate in recovery decisions.
- **3. Rigorous Multi-Tier Memory:** Context is cleanly divided across short-term, working, episodic (SQLite), semantic (vault), and project state, preventing context window bloat and eliminating prompt contamination.
- **4. Hardware-Aware Voice Design:** True barge-in cutoff and PTT hotkey control eliminate the frustrating latency and audio-feedback loops typical of conversational AI.
- **5. Comprehensive Automated Test Discipline:** 220 dedicated unit and integration tests passing with 0 failures, covering all edge cases, path traversal attacks, and permission denials.

Section 13 — Current Limitations and Open Issues

A rigorous, transparent technical assessment requires documenting current defects, operational gaps, and unverified boundaries.

Issue / Limitation	Severity	Affected Module	Impact & Practical Operational Consequence
Canonical Idempotency Divergence	**HIGH**	jarvis/tasks/recovery.py	15 tools resolve to UNKNOWN due to name mismatches (read_file vs filesystem_read) or missing entries. Blocks transient retry on read tools.
Mock-Only Recovery Tests	**MEDIUM**	tests/test_task_recovery.py	Tests used mock tool names ('filesystem_read') rather than real registered tools, masking the canonical table divergence.
No Natural Language Cancellation	**MEDIUM**	jarvis/tasks/manager.py	Cancellation requires setting task.cancellation_requested; voice barge-in 'JARVIS stop' task cancellation is deferred to Phase 8.6.
No Dynamic Task Planning	**MEDIUM**	jarvis/tasks/ (Phase 8.7)	Tasks currently require predefined steps; autonomous multi-step plan generation via Gemini is deferred to Phase 8.7.
No Autonomous Execution Loop	**MEDIUM**	jarvis/tasks/ (Phase 8.8)	End-to-end autonomous execution loop driving prompts to completed tasks belongs to Phase 8.8.

Issue / Limitation	Severity	Affected Module	Impact & Practical Operational Consequence
External Gemini Cloud Dependency	MEDIUM	jarvis/models/gemini.py	Requires active internet and valid Google Gemini API credentials. (Simulation fallback is available for CLI only).
WebSockets Deprecation Warnings	LOW	jarvis/visualizer/server.py	2 upstream warnings: websockets.legacy and WebSocketServerProtocol deprecated in Python 3.13 / Uvicorn.
Windows Win32 GUI Race Variation	LOW	jarvis/tools/screen.py	Foreground window focus timings vary under heavy workstation GPU/CPU load; handled by Scenario 2 settling delay.

Section 14 — Future Feature Roadmap

The roadmap for JARVIS is organized into five structured priority tracks (A through E). Each proposal defines clear technical approaches, dependencies, risks, complexity estimates, and acceptance criteria.

Priority A — Reliability and Safety (Immediate Baseline)

A1. Canonical Idempotency Alignment & Registry Test

- **Technical Approach:** Align _CANONICAL_IDEMPOTENCY_TABLE keys in recovery.py with exact registered tool names. Add automated unit test iterating over default_tool_registry asserting 0 tools resolve to UNKNOWN.
- **Complexity & Timing:** Estimated Complexity: High | Target Sequence: Immediate (Phase 8.5 Gate)
- **Acceptance Criteria:** Zero tools resolve to UNKNOWN; read tools retryable; all 220+ tests pass.

A2. Audit Log Integrity & Rotation

- **Technical Approach:** Implement rotating file handler for logs/audit.log with HMAC cryptographic hashing to guarantee tamper-evident audit trails.
- **Complexity & Timing:** Estimated Complexity: Medium | Target Sequence: Phase 8.5 Post-Gate
- **Acceptance Criteria:** Log size capped at 10MB; SHA-256 HMAC chain verified.

A3. Unified Tool Execution Budgeting

- **Technical Approach:** Ensure secondary inspection tools called internally by StepVerifier count toward task cumulative tool call limits.
- **Complexity & Timing:** Estimated Complexity: Low | Target Sequence: Phase 8.5 Post-Gate
- **Acceptance Criteria:** All verification queries increment task.total_tool_calls.

Priority B — Task Orchestration & Workflow Intelligence

B1. Phase 8.6: Natural-Language Task Cancellation

- **Technical Approach:** Enable voice barge-in and conversational commands (e.g., 'JARVIS, abort task') to deterministically trigger task.request_cancellation().
- **Complexity & Timing:** Estimated Complexity: Medium | Target Sequence: Phase 8.6
- **Acceptance Criteria:** Spoken abort signal halts step execution within <= 100ms with zero extra tool calls.

B2. Phase 8.7: Gemini Autonomous Task Decomposition

- **Technical Approach:** Enable Gemini 3.8 Flash to decompose ambiguous user goals into ordered, validated TaskStep sequences with assigned verification methods.
- **Complexity & Timing:** Estimated Complexity: High | Target Sequence: Phase 8.7
- **Acceptance Criteria:** Prompt generates Pydantic Task with structured steps, parameter schemas, and verification strategies.

B3. Phase 8.8: Autonomous Multi-Step Execution Loop

- **Technical Approach:** Drive tasks autonomously from goal formulation through sequential step dispatch, verification, bounded recovery, and completion.
- **Complexity & Timing:** Estimated Complexity: High | Target Sequence: Phase 8.8
- **Acceptance Criteria:** Full end-to-end task runs autonomously with zero human intervention on Low/Safe tools.

B4. Phase 8.9: Dedicated Prompt-Injection Defense Framework

- **Technical Approach:** Implement formal taint-tracking isolating untrusted screen text, webpage HTML, and document content from model instructions.
- **Complexity & Timing:** Estimated Complexity: High | Target Sequence: Phase 8.9
- **Acceptance Criteria:** Adversarial webpage strings cannot hijack agent execution or trigger unauthorized tool calls.

B5. Phase 8.10: HUD Interactive Task Progression Panels

- **Technical Approach:** Extend Cybernetic HUD with real-time visual workflow DAGs, active step indicators, and graphical approval buttons.
- **Complexity & Timing:** Estimated Complexity: Medium | Target Sequence: Phase 8.10
- **Acceptance Criteria:** HUD displays live task graph with interactive click-to-approve permission modals.

Priority C — Productivity & Daily Assistance

C1. Local Document Semantic Search

- **Technical Approach:** Integrate local document parsing (PDF, DOCX, TXT) with Gemini text-embedding-004 for instant desktop research synthesis.
- **Complexity & Timing:** Estimated Complexity: Medium | Target Sequence: Phase 9.1
- **Acceptance Criteria:** Natural language query retrieves relevant text excerpts from local workspace files.

C2. Workspace Calendar & Schedule Integration

- **Technical Approach:** Connect to local calendar (.ics) and system scheduler for proactive task reminders and scheduled executions.
- **Complexity & Timing:** Estimated Complexity: Medium | Target Sequence: Phase 9.2
- **Acceptance Criteria:** Assistant alerts user 5 minutes prior to calendar events via voice and HUD notification.

Priority D — Intelligence & Usability

D1. Dynamic Speech Rate & Audio Normalization

- **Technical Approach:** Add real-time audio volume normalization and user-configurable speech playback rates (1.0x to 1.5x) in PygameAudioOutput.
- **Complexity & Timing:** Estimated Complexity: Low | Target Sequence: Phase 10.1
- **Acceptance Criteria:** User can adjust voice speed via CLI or HUD settings without pitch distortion.

D2. Local LLM Fallback (Ollama / LLaMA-3)

- **Technical Approach:** Provide local offline model provider using Ollama when internet connectivity is severed.
- **Complexity & Timing:** Estimated Complexity: High | Target Sequence: Phase 10.2
- **Acceptance Criteria:** System transitions to OllamaProvider on network failure; basic text and tools work offline.

Priority E — Observability, Packaging & Deployment

E1. Automated Security Regression CI Suite

- **Technical Approach:** Build dedicated GitHub Actions workflow running path-traversal, prompt-injection, and secret-leakage tests on every commit.
- **Complexity & Timing:** Estimated Complexity: Medium | Target Sequence: Phase 11.1
- **Acceptance Criteria:** Automated CI blocks PRs if secrets are detected or permission tests fail.

E2. Standalone Windows Desktop Installer

- **Technical Approach:** Package JARVIS into a one-click Windows installer (PyInstaller / InnoSetup) bundling Python runtime and dependencies.
- **Complexity & Timing:** Estimated Complexity: High | Target Sequence: Phase 11.2
- **Acceptance Criteria:** Clean installer deploys JARVIS to Program Files with desktop shortcut and service manager.

Section 15 — Recommended Next Steps

To maintain rigorous architectural discipline and ensure project stability, the following dependency-aware sequence must be executed prior to authorizing any new phases:

Step 1: Correct Phase 8.5 Canonical Idempotency Table

Modify jarvis/tasks/recovery.py to align all 15 discrepant tool names (e.g. map read_file, write_file, delete_file, capture_screenshot, etc.) and explicitly classify all 37 registered tools.

Step 2: Implement Registry-Wide Automated Unit Tests

Add tests in tests/test_task_recovery.py that dynamically import default_tool_registry and assert that every registered tool resolves to either IDEMPOTENT or NON_IDEMPOTENT, strictly rejecting UNKNOWN.

Step 3: Execute Full Regression Test Suite

Run pytest tests/ -v and confirm all 220+ tests pass with zero failures.

Step 4: Conduct Code-Level Re-Audit & Formal Acceptance

Perform an independent code-level audit confirming all Phase 8.5 invariants. Obtain explicit user authorization for Phase 8.5 completion.

Step 5: Initiate Phase 8.6 Research & Architecture

Begin architectural planning for Phase 8.6 (Natural-Language Task Cancellation & Interruption). Under no circumstances should Phase 8.6 code be written before Phase 8.5 acceptance.

Section 16 — Project Directory Structure

The following sanitized ASCII tree reflects the actual directory layout of the JARVIS repository, excluding virtual environments, caches, compiled binaries, and private data.

```
D:\my work\Jarves gemini\
├── main.py                    # Primary interactive CLI text interface
├── cli_voice.py               # Gemini Live Push-to-Talk voice interface (Phase 4)
├── run_hud.py                 # Standalone Cybernetic HUD presentation runner (Phase 5)
├── pyproject.toml             # Project dependencies and build configuration
├── requirements.txt           # Pinned Python package dependencies
├── .env.example               # Safe sanitized template for environment configuration
│
├── jarvis/                    # Core production package
│   ├── config.py              # Pydantic Settings management and .env parsing
│   ├── events.py              # Asynchronous Pub/Sub EventBus and standardized EventType
│   ├── core/                  # Central orchestration and operational state
│   │   ├── agent.py           # JarvisAgent reasoning coordinator & tool loop
│   │   └── state.py           # StateMachine and AgentState lifecycle definitions
│   ├── models/                # Model provider abstractions
│   │   ├── base.py            # AIProvider and ModelResponse abstract contracts
│   │   └── gemini.py          # Google GenAI SDK wrapper (3.8 Flash, 3.6 Flash, Vision)
│   ├── security/              # Security and governance boundaries
│   │   └── permissions.py     # 5-Tier PermissionController, human gate & AuditLogger
│   ├── memory/                # 5-Tier Knowledge and Context Memory
```

```
│   │   ├── manager.py          # Unified MemoryManager coordinator
│   │   ├── short_term.py       # Ephemeral token-budgeted conversational buffer
│   │   ├── working.py          # Active task scratchpad state
│   │   ├── episodic.py         # SQLite persistent event timeline
│   │   ├── semantic.py         # JSON/Markdown persistent facts vault
│   │   └── project.py          # Git repository and workspace continuity scanner
│   ├── voice/                  # Live audio streaming and voice subsystem
│   │   ├── base.py             # Voice contracts and VoiceConfig
│   │   ├── audio_io.py         # Pygame audio capture (16kHz) and playback (24kHz)
│   │   ├── live_session.py     # Gemini Live bidirectional WebSocket session
│   │   ├── loop.py             # Main voice loop & fallback orchestration
│   │   ├── ptt.py              # Push-to-Talk global hotkey controller (Ctrl+Space)
│   │   └── tts.py              # Pyttsx3 local offline text-to-speech engine
│   ├── visualizer/             # Cybernetic HUD presentation tier
│   │   ├── server.py           # FastAPI & Uvicorn WebSocket server (127.0.0.1:8000)
│   │   ├── adapter.py          # VisualizerAdapter converting EventBus to telemetry
│   │   └── static/             # HTML5, CSS3, and JavaScript HUD dashboard assets
│   ├── tasks/                  # Structured Task & Workflow Engine (Phase 8)
│   │   ├── model.py            # Task, TaskStep, TaskStatus, StepStatus data models
│   │   ├── manager.py          # TaskManager lifecycle engine and single-active invariant
│   │   ├── executor.py         # StepExecutor non-recursive O(1) step dispatcher
│   │   ├── verifier.py         # StepVerifier multi-tier sensory evidence verifier
│   │   └── recovery.py         # RecoveryPolicy, tri-state idempotency & AntiLoopGuard
│   └── tools/                  # 37 Authoritative Standardized Tools
│       ├── base.py             # BaseTool and ToolResult abstractions
│       ├── registry.py         # ToolRegistry discovery and authorized execution engine
│       ├── filesystem.py       # read_file, write_file, delete_file, list_dir, search_files
│       ├── terminal.py         # TerminalExecuteTool safe command executor
│       ├── applications.py     # launch_application, focus_app, close_app, terminate_process
│       ├── computer.py         # mouse_click, keyboard_type, press_key, capture_screenshot
│       ├── screen.py           # capture_screen (MSS), get_active_window, analyze_screen
│       ├── browser.py          # browser_open, browser_read_page, browser_control
│       ├── web_research.py     # web_search, fetch_web_content
│       ├── camera.py           # open_camera, capture_camera_frame, analyze_camera_frame
│       └── media.py            # youtube_control, music_control
│
├── tests/                      # 220 Automated Unit & Integration Tests
│   ├── test_tasks.py           # Phase 8.1 Task Model Tests (16 tests)
│   ├── test_task_manager.py    # Phase 8.2 Task Manager Tests (16 tests)
│   ├── test_task_executor.py   # Phase 8.3 Step Executor Tests (12 tests)
│   ├── test_step_verifier.py   # Phase 8.4 Step Verifier Tests (15 tests)
│   ├── test_task_recovery.py   # Phase 8.5 Recovery & Anti-Loop Tests (24 tests)
│   ├── test_tools.py           # Phase 2 Tools & Permission Tests (8 tests)
│   ├── test_voice.py           # Phase 4 Live Voice Tests (27 tests)
│   ├── test_visualizer.py      # Phase 5 Visualizer HUD Tests (14 tests)
│   ├── test_memory.py          # Phase 3 Memory Subsystem Tests (28 tests)
│   ├── test_phase6.py          # Phase 6 Real-World App Tests (30 tests)
│   └── test_phase7.py          # Phase 7 Vision & Grounding Tests (30 tests)
│
└── docs/                       # Architectural Specifications & Reports
    ├── architecture.md         # Master architecture reference
    ├── phase8-architecture.md  # Phase 8 task workflow formal specifications
    ├── phase8-implementation-progress.md # Chronological progress tracker
    └── project_report/         # Published official technical reports
        ├── architecture_diagram.png    # High-resolution architectural graphic
```

```
        ├── architecture_diagram.mermaid # Plain-text Mermaid specification
        └── JARVIS_Complete_Project_Technical_Report.docx # Primary official report
```

Section 17 — Setup and Operation Guide

This guide outlines the verified instructions for deploying, configuring, and operating JARVIS on a local workstation.

17.1 System Prerequisites

- **Operating System:** Windows 10 / Windows 11 (64-bit recommended for Win32 API window hooks).
- **Python Environment:** Python 3.11, 3.12, or 3.13 (Python 3.13.5 confirmed in production).
- **Hardware Audio Devices:** Standard microphone and speaker/headphone configured as Windows default audio devices.
- **Network Connectivity:** Outbound HTTPS (port 443) access to generativelanguage.googleapis.com.

17.2 Installation & Dependency Setup

```
# 1. Clone repository and navigate to workspace
cd "D:\my work\Jarves gemini"

# 2. Create and activate a clean virtual environment
python -m venv .venv
.venv\Scripts\activate

# 3. Install pinned dependencies
pip install --upgrade pip
pip install -r requirements.txt
```

17.3 Safe Environment Configuration

Copy .env.example to .env and configure your Google Gemini API key. Never commit .env to version control:

```
# --- Google Gemini API Configuration ---
GEMINI_API_KEY=[YOUR_ACTUAL_GEMINI_API_KEY_HERE]
GEMINI_PRIMARY_MODEL=gemini-3.8-flash
GEMINI_FAST_MODEL=gemini-3.6-flash
GEMINI_EMBEDDING_MODEL=text-embedding-004

# --- Assistant Persona & Identity ---
JARVIS_NAME=JARVIS
USER_NAME=Sir

# --- Security Governance ---
PERMISSION_CONFIRMATION_THRESHOLD=3
LOG_LEVEL=INFO
```

```
# --- Voice Subsystem Settings ---
VOICE_ENABLED=true
GEMINI_LIVE_VOICE=Charon

# --- Cybernetic HUD Presentation ---
WEB_HUD_ENABLED=true
WEB_HUD_HOST=127.0.0.1
WEB_HUD_PORT=8000
WEB_HUD_AUTO_OPEN=true
```

17.4 Running the Assistant

```
# Mode 1: Interactive CLI Text Interface (with simulation mode if no API key)
python main.py

# Mode 2: Push-to-Talk Voice Interface (Hold [Ctrl+Space] to speak)
python cli_voice.py

# Mode 3: Voice Interface + Auto-Launched Cybernetic HUD
python cli_voice.py --hud

# Mode 4: Standalone HUD Presentation Server (Observability development)
python run_hud.py

# Mode 5: Execute Automated Regression Test Suite
pytest tests/ -v
```

Section 18 — Glossary of Technical Terms

Term / Concept	Formal Technical Definition
AI Agent	An autonomous computational entity utilizing language models for reasoning and tool invocation to satisfy complex goals.
Gemini Live Preview	Google's bidirectional streaming API enabling low-latency full-duplex audio and voice communication over WebSockets.
Tool Calling	The protocol by which an LLM emits structured JSON parameters requesting the host application execute a deterministic function.
Permission Controller	A centralized governance layer evaluating the risk classification of an action and enforcing mandatory human authorization.
Event Bus	An asynchronous Pub/Sub message broker enabling decoupled communication across agent, tools, visualizer, and state engines.

Term / Concept	Formal Technical Definition
Working Memory	An ephemeral, task-scoped scratchpad retaining intermediate hypotheses, active goals, and partial outputs.
Episodic Memory	A structured, timeline-based persistence engine (SQLite) recording historical user interactions, tool executions, and system events.
Semantic Memory	A persistent knowledge repository storing discrete facts, user preferences, and profile data in categorized key-value vaults.
Project Memory	A workspace awareness engine inspecting local Git branches, commit logs, and repository directory trees to maintain context.
Task Lifecycle	The formal progression of a multi-step goal through PENDING, PLANNING, EXECUTING, VERIFYING, COMPLETED, FAILED, and CANCELLED.
Step Verifier	A post-execution sensory validation engine evaluating real-world OS/visual evidence to confirm an intended outcome occurred.
Tri-State Idempotency	A three-valued safety policy classifying operations as IDEMPOTENT (safe to repeat), NON_IDEMPOTENT (mutating), or UNKNOWN.
Anti-Loop Guard	A structural safety mechanism computing SHA-256 action fingerprints to detect and halt recurring failures and A-B-A-B cycle loops.
Hardware Barge-In	The immediate halting of active audio output playback upon detecting user speech input, preventing audio feedback loops.
Prompt Injection	A vulnerability where untrusted external data (screen OCR, web text) contains adversarial instructions attempting to hijack the agent.

Section 19 — Conclusion

The JARVIS project represents a substantial, highly disciplined engineering achievement in the personal AI assistant domain. By enforcing strict boundaries between probabilistic reasoning and deterministic execution, establishing a rigorous 5-tier security permission model, implementing comprehensive multi-tier memory, and building an evidence-based task verification and bounded recovery engine, the repository establishes a robust foundation for safe desktop automation.

The Phase 8.5 audit finding regarding canonical idempotency table alignment highlights the indispensable value of rigorous code-level governance. Because JARVIS defaults to fail-safe behavior (UNKNOWN tools strictly prohibited from automatic retry), the system's operational safety remains uncompromised. Correcting

the canonical table and reinforcing test coverage against the live registry will conclude Phase 8.5 and pave the way for natural-language cancellation (Phase 8.6) and autonomous multi-step planning (Phase 8.7).